Name: Joshua Bourton User-name: xtzv61
Algorithm A: IA (A* with iterative deepening)
Algorithm B: BG (Basic Greedy Search)
Description of enhancement of Algorithm A:

Note: Both Enhancement 1 and 2 involved modification of the original prims_heuristic() function. This function was subsequently renamed to pruned_prims_heuristic() to reflect its enhanced functionality.

Enhancement 1: The final city in the minimum spanning tree (MST) (calculated from city zero) was used as the starting city, as opposed to a randomly generated one. This was done by modifying the algorithm used for calculating the MST heuristic value from a given node so that it also had the ability to return the MST itself. A boolean value was used to inform the function whether to return the MST itself or its heuristic value.
The MST connects cities in a way that minimises the path cost, so selecting this final city helped provide a consistently low score for all ten provided city files.

Enhancement 2: Heap pruning was used on the MST heuristic calculation to skip over unnecessary computations.
This was done by checking that the city was already present in the MST prior to and skipping over the rest of the loop if it was.

Description of enhancement of Algorithm B:

DESCRIPTION OF ALGORITHM ONLY IF THE ALGORITHM IS NOT COVERED IN LECTURES

Enhancement 1A: To give the greedy algorithm a greater chance of returning a higher quality tour, a *multiple_restarts()* function was constructed that ran multiple iterations of the algorithm and returned the best tour found from amongst those run-throughs. To prevent the same starting cities being used for each iteration, the first ten edges in the sorted edge set in the *Greedy_TSP()* function were randomly shuffled.

Enhancement 1B: To prevent runtime from going beyond reasonable bounds, a function *select_num_restarts()* was written to select the number of times the *multiple_restarts()* function is allowed to run the greedy algorithm. It returns this value based on predefined numbers chosen from knowledge of how long the greedy algorithm takes to execute on the ten provided tours .

Enhancement 2: Inspired by the paper by Chaing, et al., (2016) (DOI here: http://dx.doi.org/10.1109/ICMLC.2016.7872949), Algorithm B was enhanced by adding a function to implement the 2-OPT technique, which modified the global *tour* variable to obtain an enhanced tour. 2-OPT works by iterating over all the pairs edges in the tour and examining whether reversing the order of two cities in the tour would result in a tour with a lower path cost. If it would, the tour is subsequently modified to reflect the swap. Combined with enhancments 1A and 1B, this greatly improved tour quality, with the quality of tours found from the enhanced greedy algorithm now being on a similar level to the quality of tours discovered with the iterative depeening A* search.

Description of *non-standard* Algorithm A:

Describe any non-standard algorithms you have implemented that ***have not been covered in lectures*** *(otherwise these boxes should be blank) You need to convince me that your implementation is indeed that of the named algorithm and you need to* ***provide a full reference to the source for your algorithm****. You should* ***include a pseudocode description****. You can vary the sizes of these boxes but do not change the font (Calabri), font size (11), the paragraph properties (single space) or the header and footer and everything should fit onto one side of A4. (You can delete these instructions.)*

Remember: You need my express permission to implement a non-standard algorithm!

Description of *non-standard* Algorithm B:

Type here, as above.